Testing Python with unittest

Test cases, fixtures, and controlled dependencies

CS413

October 8, 2026

Python's built-in testing framework

- `unittest` comes with Python. No separate installation is needed.
- Tests are methods on subclasses of `unittest.TestCase`.
- A runner executes the tests and reports their outcomes.

```
python3 -m unittest -v
```

An executable contract

- For a stack, the last item pushed should be the first item popped.
- Popping an empty stack should raise `IndexError`.
- Tests turn these expectations into repeatable checks.
- Passing examples provide evidence. They do not prove correctness for every input.

The stack under test

```python
# stack.py
class Stack:
    def __init__(self):
        self._items = []

    def push(self, value):
        self._items.append(value)

    def pop(self):
        if not self._items:
            raise IndexError("empty stack")
        return self._items.pop()
```

A first TestCase

```python
# test_stack.py
import unittest
from stack import Stack

class TestStack(unittest.TestCase):
    def test_pop_returns_last_pushed(self):
        stack = Stack()
        stack.push("x")
        stack.push("y")
        self.assertEqual(stack.pop(), "y")

if __name__ == "__main__":
    unittest.main()
```

The guard enables direct execution without running the suite on import.

Running the examples

```
cd examples/unittest_demo
python3 -m unittest -v
python3 -m unittest test_stack
python3 -m unittest test_stack.TestStack
python3 -m unittest test_stack.TestStack.test_empty_pop
python3 test_stack.py -v
```

The last command uses the `unittest.main()` guard.
All example commands run from this demo directory.

Discovery and selection

```
python3 -m unittest discover -s . -p 'test*.py' -v
python3 -m unittest -k pop
python3 -m unittest -f
python3 -m unittest -b
```

- Default discovery uses `test*.py` files and methods starting with `test`.
- `-k` matches a substring or wildcard pattern, not a pytest Boolean expression.
- `-f` stops at the first failure or error. `-b` buffers output.
- For nested test directories, use importable packages with `__init__.py`.

Failures and errors

Suppose `pop()` incorrectly calls `self._items.pop(0)`.

```
FAIL: test_pop_returns_last_pushed (...)
...
AssertionError: 'x' != 'y'
- x
+ y
```

- A failed assertion reports a failure.
- An unexpected exception, such as `NameError`, reports an error.
- Read the failing expectation and traceback before changing the code.

Assertion methods express the expectation

Method	Meaning
`assertEqual(a, b)`	Equal values
`assertIs(a, b)`	The same object
`assertIsNone(x)`	The value is `None`
`assertIn(x, xs)`	Membership
`assertTrue(p)`	Truthiness
`assertCountEqual(a, b)`	Same elements and multiplicities

For equality, `assertEqual(a, b)` usually gives a clearer diagnostic than `assertTrue(a == b)`.

Values, identity, and order

```
def test_list_behavior(self):
    a = [1, 2]
    b = [1, 2]
    self.assertEqual(a, b)
    self.assertIsNot(a, b)
    self.assertCountEqual([1, 2, 2], [2, 1, 2])
```

- Choose the assertion from the contract.
- A stack test needs order-sensitive equality.
- Use `assertCountEqual` only when ordering is irrelevant.

Expected exceptions

```
def test_empty_pop(self):
    stack = Stack()
    with self.assertRaisesRegex(IndexError, "empty stack"):
        stack.pop()
```

- `assertRaises` checks the exception type.
- `assertRaisesRegex` also searches the message with a regular expression.
- No exception or a different exception makes the test unsuccessful.
- Keep unrelated setup outside the context manager.

Fresh state with setUp

```
class TestStack(unittest.TestCase):
    def setUp(self):
        self.stack = Stack()

    def test_round_trip(self):
        self.stack.push(7)
        self.assertEqual(self.stack.pop(), 7)

    def test_empty_pop(self):
        with self.assertRaises(IndexError):
            self.stack.pop()
```

Each test gets a new `TestCase` instance and its own `setUp()` call.

Fixture lifecycle

- For a normal test: `setUp()`, test method, `tearDown()`, cleanups.
- `tearDown()` runs after a failing test if `setUp()` succeeded.
- If `setUp()` raises, the test and `tearDown()` do not run.
- Registered `addCleanup` callbacks still run if setup fails.
- Cleanup callbacks run in reverse registration order.

Temporary resources and cleanup

```
import tempfile
from pathlib import Path

class TestFiles(unittest.TestCase):
    def setUp(self):
        temp = tempfile.TemporaryDirectory()
        self.addCleanup(temp.cleanup)
        self.directory = Path(temp.name)

    def test_text_round_trip(self):
        path = self.directory / "out.txt"
        path.write_text("hello", encoding="utf-8")
        self.assertEqual(path.read_text(encoding="utf-8"),
                         "hello")
```

Register cleanup immediately after acquiring the resource.

Shared fixtures and test independence

- `setUpClass` and `tearDownClass` manage class-level resources.
- Use `@classmethod` for these methods.
- A shared mutable resource can leak state between tests.
- Prefer per-test setup until the cost justifies sharing.
- Each test should pass when run alone. Tests must not depend on their execution order.

Multiple cases with subTest

```
def test_lifo_cases(self):
    for values in ([], [7], [2, 5, 2]):
        with self.subTest(values=values):
            stack = Stack()
            for value in values:
                stack.push(value)
            actual = [stack.pop() for _ in values]
            self.assertEqual(actual, list(reversed(values)))
```

- A failing subtest reports its parameters and normally allows the loop to continue.
- These are subcases of one test method, not separately discovered methods.
- Construct fresh state inside each subtest.

Choosing useful cases

- An empty stack exercises the error contract.
- Distinct values expose FIFO behavior accidentally implemented as LIFO.
- Repeated values detect implementations that discard duplicates.
- Interleaved pushes and pops exercise changes in state.
- A regression test should fail with the specific bug and pass after the fix.

Floating-point expectations

```
def test_decimal_sum(self):
    self.assertAlmostEqual(0.1 + 0.2, 0.3, delta=1e-12)
```

- `delta` sets an absolute tolerance.
- Choose the tolerance from the numerical requirements.
- The default `places=7` checks rounded differences, not significant digits.
- For a relative tolerance, use `math.isclose` with explicit tolerances.

Capturing printed output

```
from contextlib import redirect_stdout
from io import StringIO
from stack import announce

def test_announce(self):
    output = StringIO()
    with redirect_stdout(output):
        announce("Ada")
    self.assertEqual(output.getvalue(), "Hello, Ada!\n")
```

Here `announce(name)` calls `print(f"Hello, {name}!")`.
The newline is part of the expected output.

Controlling environment variables

```
# stack.py
import os

def greeting():
    return "Hello, " + os.getenv("USER_NAME", "guest")

# Inside a TestCase
from unittest.mock import patch

def test_greeting(self):
    with patch.dict("os.environ", {"USER_NAME": "Ada"}):
        self.assertEqual(greeting(), "Hello, Ada")
```

Place the imports at module scope. The context restores the environment.

Mocking an external dependency

```
# service.py
from urllib.request import urlopen

def status_code(url):
    with urlopen(url, timeout=2) as response:
        return response.status

# Inside a TestCase; patch is imported at module scope
@patch("service.urlopen", autospec=True)
def test_status_code(self, mock_open):
    mock_open.return_value.__enter__.return_value.status = 200
    self.assertEqual(status_code("https://example.org"), 200)
    mock_open.assert_called_once_with("https://example.org",
                                      timeout=2)
```

Where to patch, and what a mock proves

- Patch `service.urlopen`: that is where `status_code` looks up the name.
- Patching the original library after `service` imports the name would miss this reference.
- `autospec=True` constrains the mock to the original interface where inspectable.
- The test checks our interaction with a substitute. It does not establish that the network service works.
- Separate integration tests can exercise the real dependency.

Skipped tests and expected failures

```
@unittest.skip("requires the course server")
def test_server(self):
    ...

@unittest.expectedFailure
def test_known_bug(self):
    self.assertEqual(buggy_function(), expected)
```

- Illustrative placeholders, not part of the runnable demo.
- A skipped test does not run. An expected failure documents a known bug.
- An unexpected success makes the unittest run unsuccessful.
- Use a reason or issue reference so the marker can be revisited.

unittest and pytest: equivalent intentions

unittest	pytest
`TestCase` method	Usually a plain function
`self.assertEqual(a, b)`	`assert a == b`
`self.assertRaises(E)`	`pytest.raises(E)`
`setUp` / cleanups	Fixtures / yield cleanup
`subTest` loop	`mark.parametrize` cases
Standard library	Separately installed package

Both can express the same behavioral expectations. The setup and reporting conventions differ.

Exercise: free variables

For tuple-based expressions, the intended rules are:

$$FV(x) = \{x\}, \qquad FV(e_1\ e_2) = FV(e_1) \cup FV(e_2)$$

$$FV(\lambda x.e) = FV(e) \setminus \{x\}$$

```
("var", "x")
("app", ("var", "x"), ("var", "y"))
("lam", "x", ("var", "x"))
```

- Write tests for a bound variable and a variable that remains free.
- Add an application and a nested lambda, using `subTest`.
- Which case detects a missing subtraction in the lambda rule?

Exercise discussion

```
from expr import free_vars

class TestFreeVars(unittest.TestCase):
    def test_cases(self):
        cases = [
            (("lam", "x", ("var", "x")), set()),
            (("lam", "x", ("var", "y")), {"y"}),
            (("app", ("var", "x"), ("var", "y")), {"x", "y"}),
        ]
        for expr, expected in cases:
            with self.subTest(expr=expr):
                self.assertEqual(free_vars(expr), expected)
```

The first case detects the missing subtraction. The demo includes a nested case.

Testing workflow and references

- Write a specific expectation and observe the intended failure.
- Fix the implementation, rerun the test, then run the suite.
- Check that the reported test count is plausible. An empty suite proves nothing.
- Demo: `examples/unittest_demo/`.

Official Python documentation:

- unittest: cases, assertions, fixtures, and discovery
- unittest.mock: patching and test doubles